

# Basic Android

*Android training Activitys*



# Agenda

1. **Activity Overview**
2. **Process Lifecycle**
3. **Configuration Changes**
4. **Start Activity & Get Results**
5. **Saving Persistent State**
6. **Permission**
7. **Manifest Elements**



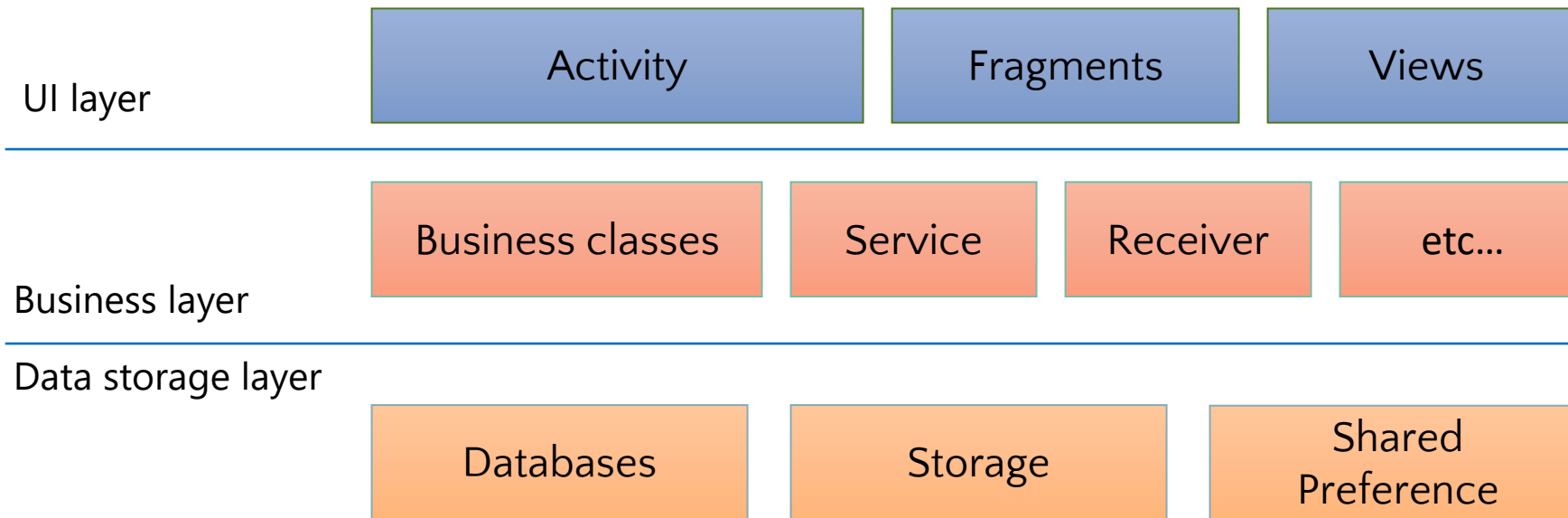
# Activity Overview

# Activity Overview

- Main building components of Android Application
- Activity is a **most major components** of Android
- Activity provides the **user interface** with which user can **interact** with **application**.
- All other UI components such as (views, fragment, etc...) rely on Activity.
- One Activity can start another Activity
- When a new activity starts, it's pushed to a **back stack** - a LIFO data structure.

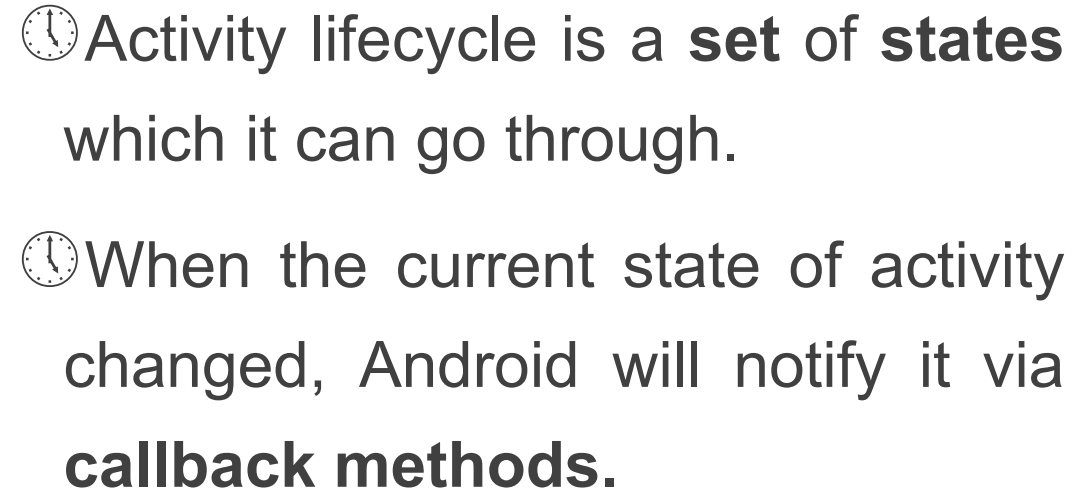
# Activity Overview (2)

- Activity in an Android application structure





# Activity Lifecycle

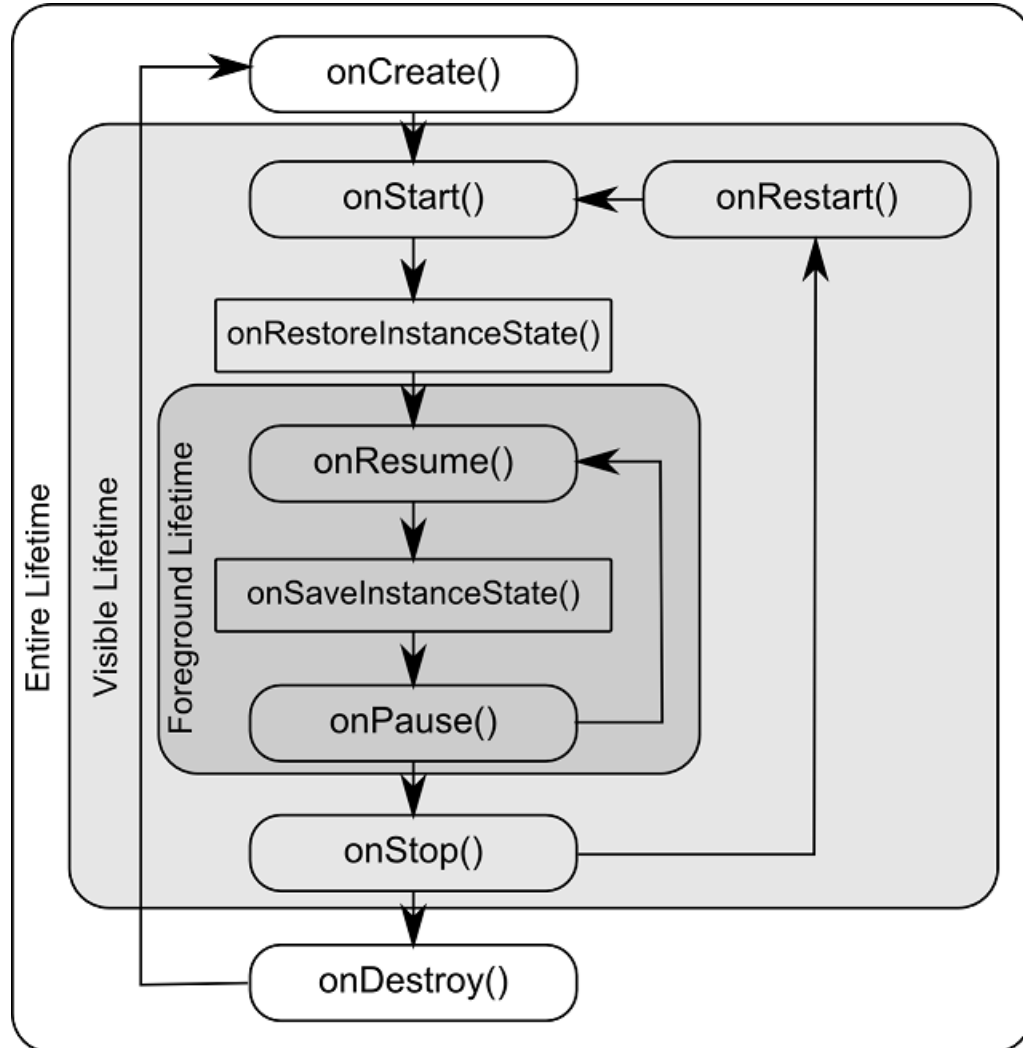


# Activity Lifecycle (2)

- Activity with lifecycle callback methods

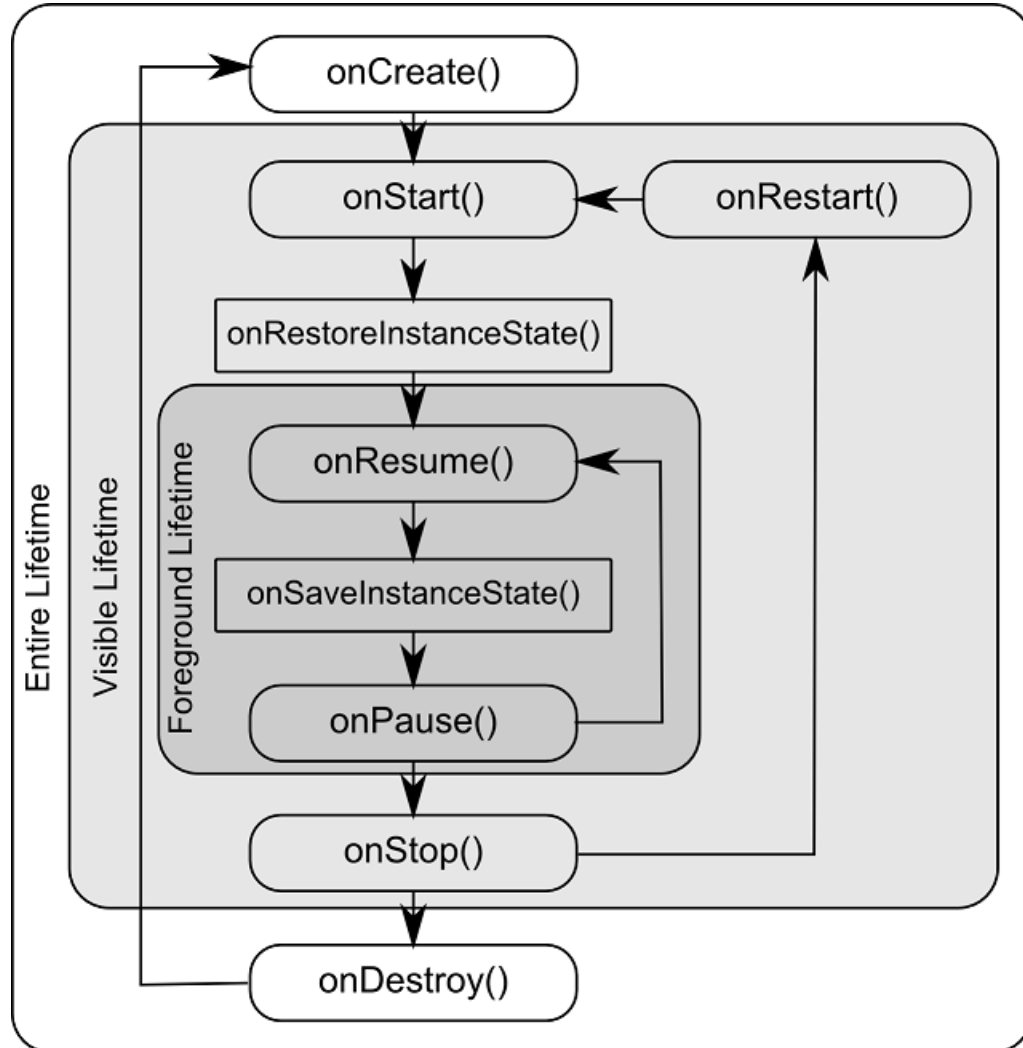
```
public class MyActivity {  
    // The activity is created  
    public void onCreate(Bundle savedInstanceState) { }  
    // The activity is about to become visible  
    protected void onStart() { }  
    //The activity is visible (it is resumed)  
    protected void onResume() { }  
    // Another activity is taking focus (it is paused)  
    protected void onPause() { }  
    // The activity is no longer visible (it is stopped)  
    protected void onStop() { }  
    // The activity is about to be destroyed  
    protected void onDestroy() { }  
}
```

# Activity Lifetime



- **Entire lifetime** from `onCreate()` to `onDestroy()`
- **Visible lifetime** is from `onStart()` to `onStop()`
- **Foreground lifetime** is from `onResume()` to `onPause()`

# Activity Lifecycle (2)



- **Entire lifetime** used to setup/release the activity's global state
- **Visible lifetime** maintain the resources needed to show activity to user. (Ex: register a Receiver)
- **Foreground lifetime** maintain the resources needed to interact with user.

# Activity's essential states

- **Running/Resumed:** the activity is in the foreground of the screen and receive user's focus.
- **Paused:** partially obscured by another activity that in the foreground and get user's focus.
- **Stopped:** completely obscured by another activity.
- **Destroyed:** drop the activity from memory by either asking it to finish, or simply killing its process.

# Process Lifecycle

- There are **four states** a process can be in based on the activities running
  - The **foreground activity**: the activity at the top of the screen that the user is currently interacting with and will only be killed as a last resort, if it uses more memory than is available on the device.
  - A **visible activity**: an activity that is visible to the user but not in the foreground, such as one sitting behind a foreground dialog or next to other activities in multi-window mode.
  - A **background activity**: an activity that is not visible to the user and has been stopped. No longer critical, the system may safely kill its process to reclaim memory for other foreground or visible processes.
  - An **empty process**: is one hosting no activities or other application components (such as Service or BroadcastReceiver classes). These are killed very quickly by the system as memory becomes low.

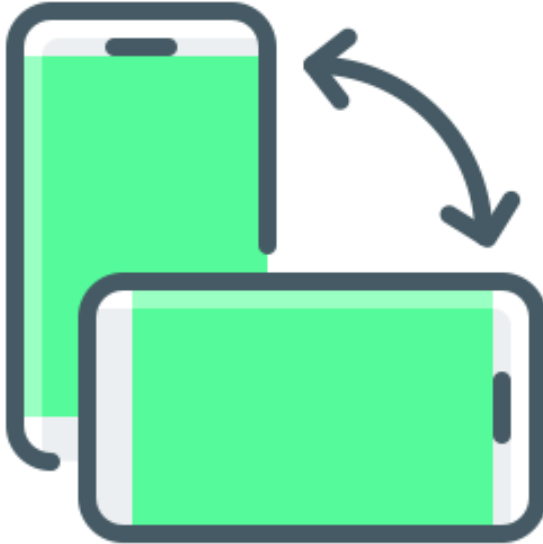
# Configuration Changes

# Configuration Changes

**Some device configurations can change while the app is running:**

- ✓ App display size
- ✓ Screen orientation
- ✓ Font size and weight
- ✓ Locale
- ✓ Dark mode versus light mode
- ✓ Keyboard availability

# Configuration Changes (2)



- The system recreates an Activity when a configuration change occurs. The system calls [onDestroy\(\)](#) and destroys the existing Activity instance.
- It then creates a new instance using [onCreate\(\)](#), and this new Activity instance is initialized with the new, updated configuration.

# Restrict Activity Recreation

- To disable activity recreation for particular configuration changes, add the configuration type to `android:configChanges` in the `<activity>` entry in `AndroidManifest.xml` file. Possible values appear in the documentation for the `android:configChanges` attribute.

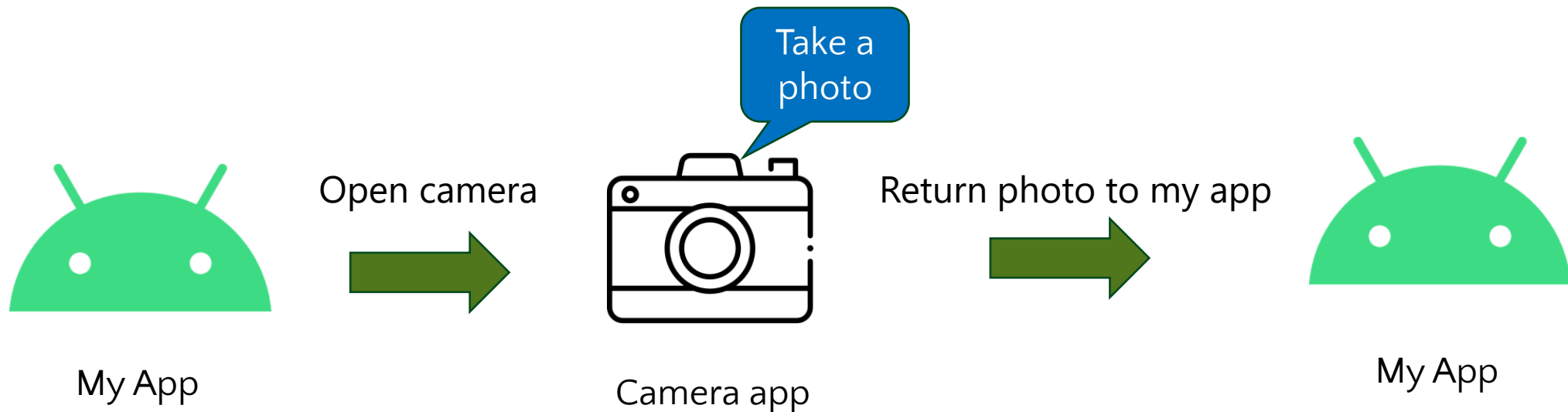
```
<activity
    android:name=".MyActivity"
    android:configChanges="orientation|screenSize|screenLayout|keyboardHidden"
    android:label="@string/app_name">
```



# Start Activity & Get Results

# Start Activity & Get Results

- Starting another activity, whether it is one within your app or from another app, doesn't need to be a one-way operation.
- We can also start an activity and receive a result back.



# Register a callback for an activity result

- When in a ComponentActivity or a Fragment, the Activity Result APIs provide a registerForActivityResult() API for registering the result callback.
- registerForActivityResult() takes an ActivityResultContract and an ActivityResultCallback and returns an ActivityResultLauncher, which using to launch the other activity.

ActivityResultContract

The APIs provide default contracts for basic intent actions like taking a picture, requesting permissions, and so on.

ActivityResultCallback

A single method interface with an onActivityResult() method that takes an object of the output type defined in the ActivityResultContract

# Example

```
class MainActivity: AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    private val myActivityResultLauncher = registerForActivityResult(
        ActivityResultContracts.StartActivityForResult()) { result ->
        if (result.resultCode == Activity.RESULT_OK) {
            // Handle the successful result
            val data: Intent? = result.data
            // ...
        } else {
            // Handle the failed result
            // ...
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Example usage of the activity result launcher
        binding.button.setOnClickListener { it: View!
            val intent = Intent( packageContext: this, SecondActivity::class.java)
            myActivityResultLauncher.launch(intent)
        }
    }
}
```

- In this example, we define a activity result launcher using **registerForActivityResult**
- In the onCreate method, we create a button and set a click listener to launch the SecondActivity using the activity result launcher.
- Once the activity finishes and returns a result, the lambda passed to the registerForActivityResult method will be called, and we can handle the result based on the resultCode

# Example

```
class SecondActivity: AppCompatActivity() {
    private lateinit var binding: ActivityMainBinding

    private val myActivityResultLauncher = registerForActivityResult(
        ActivityResultContracts.StartActivityForResult()) { result ->
        if (result != null) {
            // Handle the successful result
            // ...
        } else {
            // Handle the failed result
            // ...
        }
    }

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)

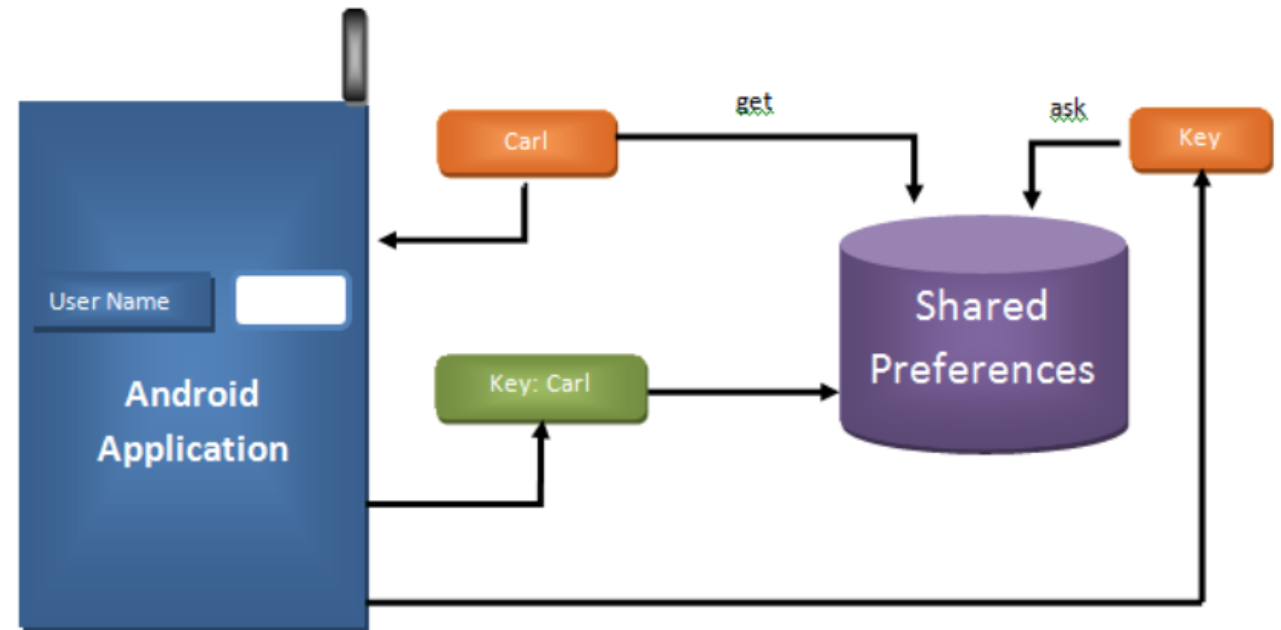
        // Example usage of the activity result launcher
        binding.button.setOnClickListener { it: View!
            myActivityResultLauncher.launch( input: "image/*")
        }
    }
}
```

- In SecondActivity, we define another activity result launcher using registerForActivityResult method
- This contract allows us to get content (e.g., an image) from another app or activity

# Saving Persistent State

# Saving Persistent State

There are two kinds of persistent state that an activity will deal with



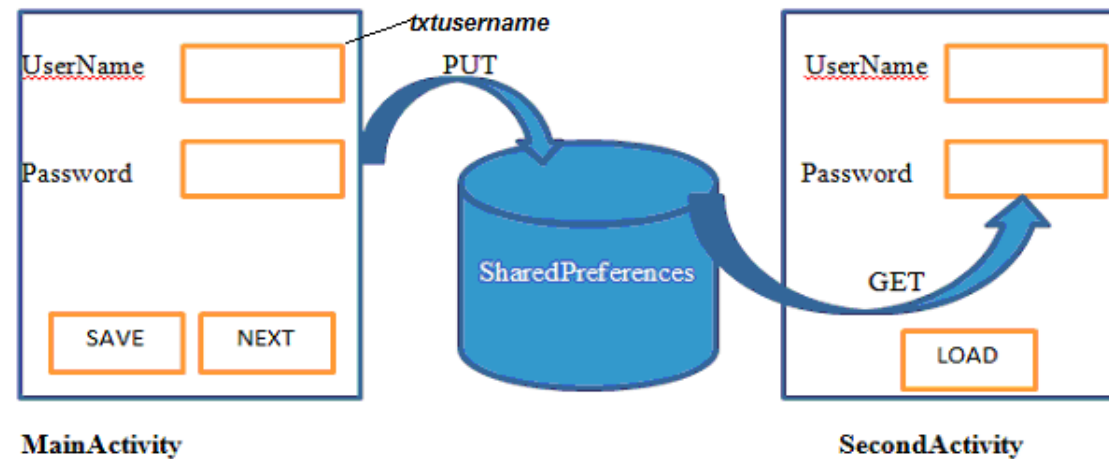
# For Content Provider

- Activities use an "edit in place" user model according to two rules:
  - ✓ When creating a new document, the backing database entry or file for it is created immediately.
  - ✓ When an activity's `onPause()` method is called, it should commit to the backing content provider or file any changes the user has made.

➡ This model is designed to prevent data loss when a user is navigating between activities, and allows the system to safely kill an activity at any time after it has been stopped.

# For Shared Preferences

- This API is provided for managing internal state associated.
  - ✓ For example, to remember the user's preferred initial display in a calendar (day view or week view) or the user's default home page in a web browser.
- Activity persistent state is managed with the method getPreferences(int), to retrieve and modify a set of name/value pairs associated with the activity.





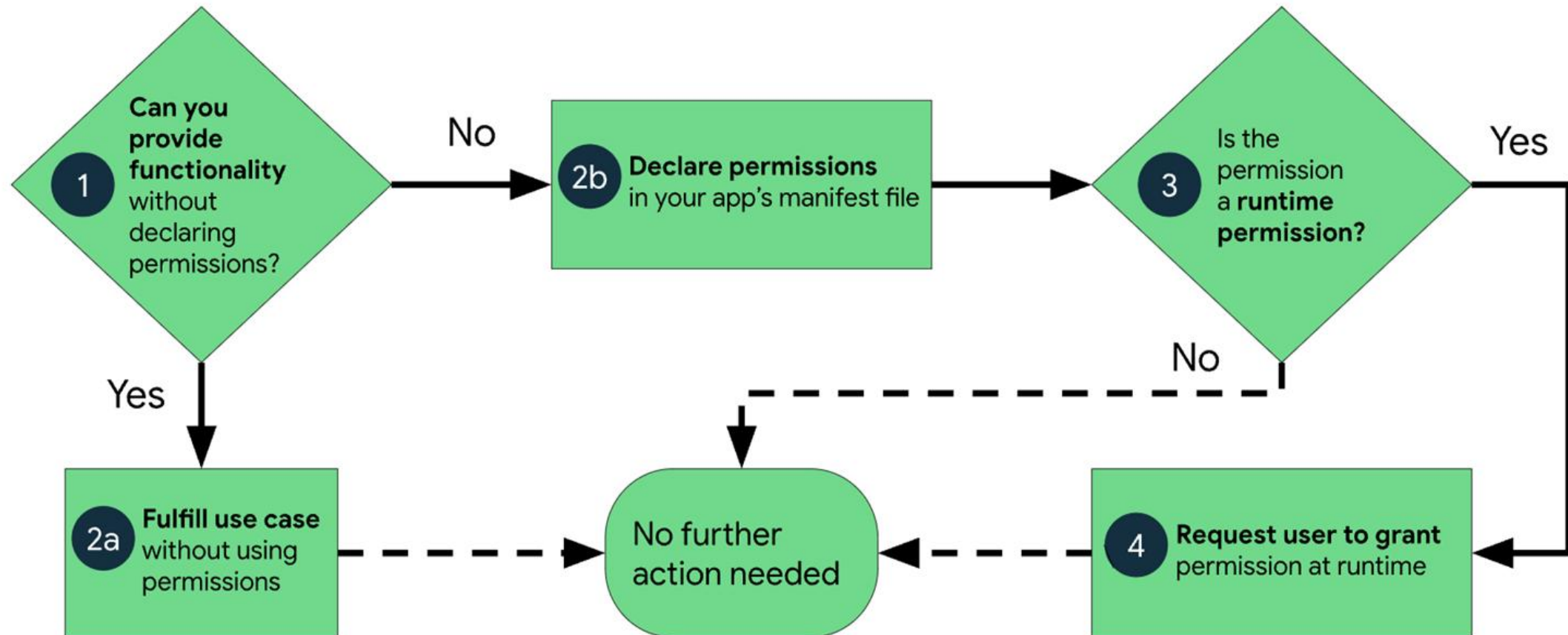
# Permission

# What is Permission ?

- In the Android operating system, **permissions are requests made by apps to access certain features or data on a device.**
- **Permissions** serve as a security mechanism to protect user privacy and prevent apps from accessing sensitive information or performing potentially harmful actions without the user's knowledge or consent.



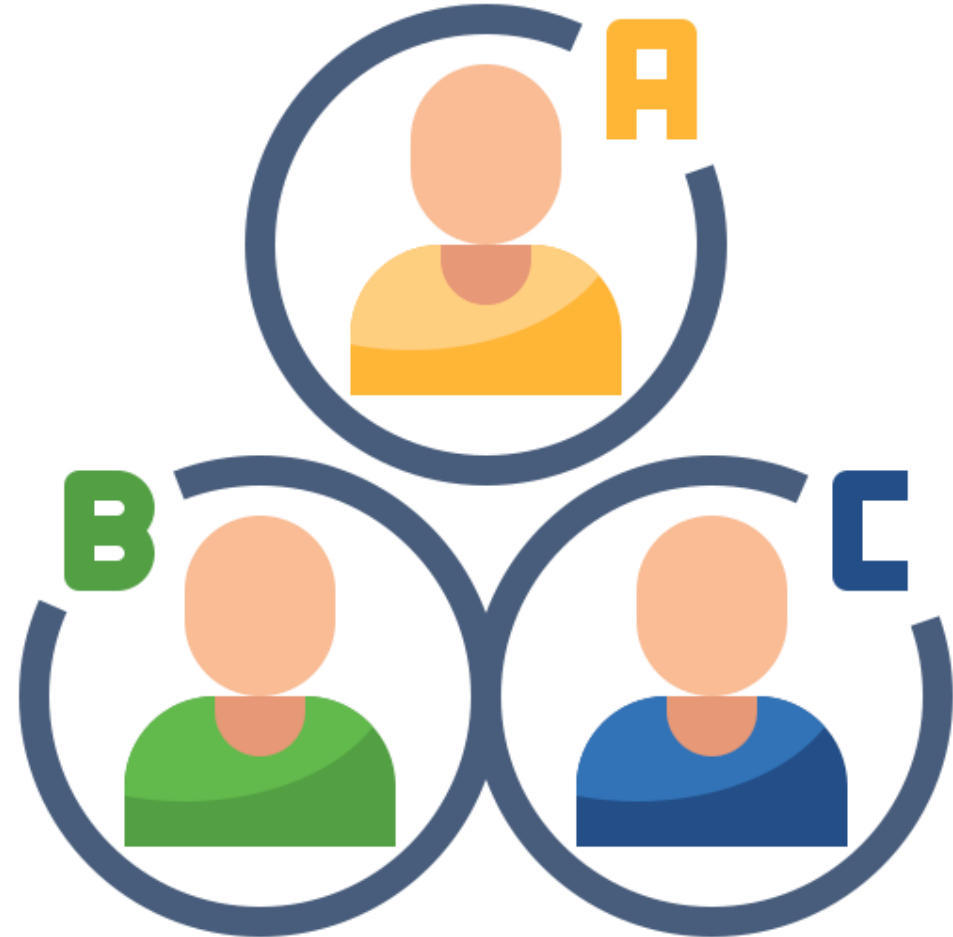
# Workflow for using permission



# Types of permission

There are 3 types of permission:

- ✓ Install-time permissions
- ✓ Runtime permissions
- ✓ Signature permissions



# Install-time permissions

- **Install-time permissions** give the app limited access to restricted data or let the app perform restricted actions that minimally affect the system or other apps.
- When declare **install-time permissions** in your app, an app store presents an install-time permission notice to the user when they view an app's details page. The system automatically grants your app the permissions when the user installs app.

Version 1.234.5 may request access to



## Other

- have full network access
- view network connections
- prevent phone from sleeping
- Play Install Referrer API
- view Wi-Fi connections
- run at startup
- receive data from Internet

# Install-time permission (2)

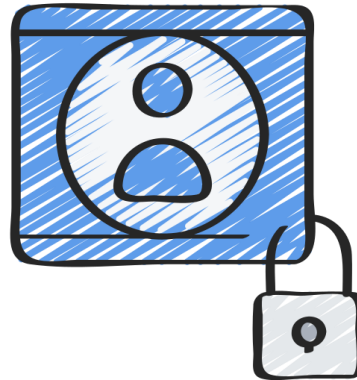
Android includes several sub-types of **install-time permissions** :

- **Normal permissions:** These permissions allow access to data and actions that extend beyond your app's sandbox but present very little risk to the user's privacy and the operation of other apps.
- **Signature permissions:** The system grants a signature permission to an app only when the app is signed by the same certificate as the app or the OS that defines the permission.

# Install-time permissions (3)

## ■ Normal permissions:

- ACCESS\_NETWORK\_STATE
- INTERNET
- ACCESS\_WIFI\_STATE
- VIBRATE
- ACCESS\_NETWORK\_STATE
- NFC
- RECEIVE\_BOOT\_COMPLETED
- WAKE\_LOCK

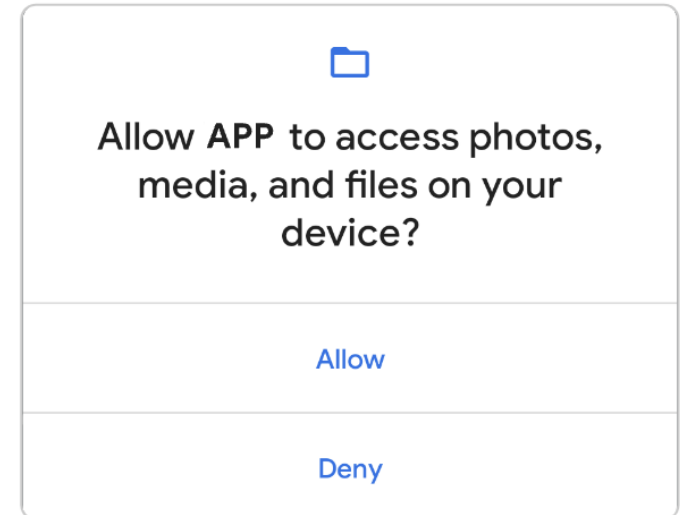


## ■ Signature permissions:

- BIND\_ACCESSIBILITY\_SERVICE
- BIND\_DEVICE\_ADMIN
- BIND\_DREAM\_SERVICE
- BIND\_INPUT\_METHOD
- BIND\_WALLPAPER

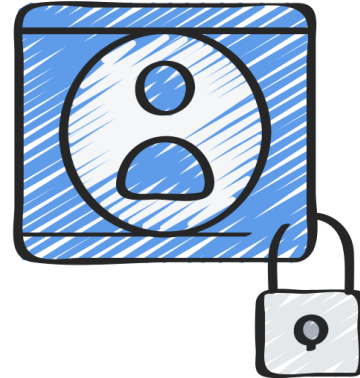
# Runtime permissions

- **Runtime permissions**, also known as dangerous permissions, give your app additional access to restricted data or let your app perform restricted actions that more substantially affect the system and other apps.
- Need to request runtime permissions in your app before you can access the restricted data or perform restricted actions.



# List Runtime permissions

- CAMERA
- READ\_CONTACTS
- WRITE\_CONTACTS
- RECORD\_AUDIO
- ACCESS\_FINE\_LOCATION
- ACCESS\_COARSE\_LOCATION
- READ\_PHONE\_STATE
- CALL\_PHONE
- READ\_CALL\_LOG



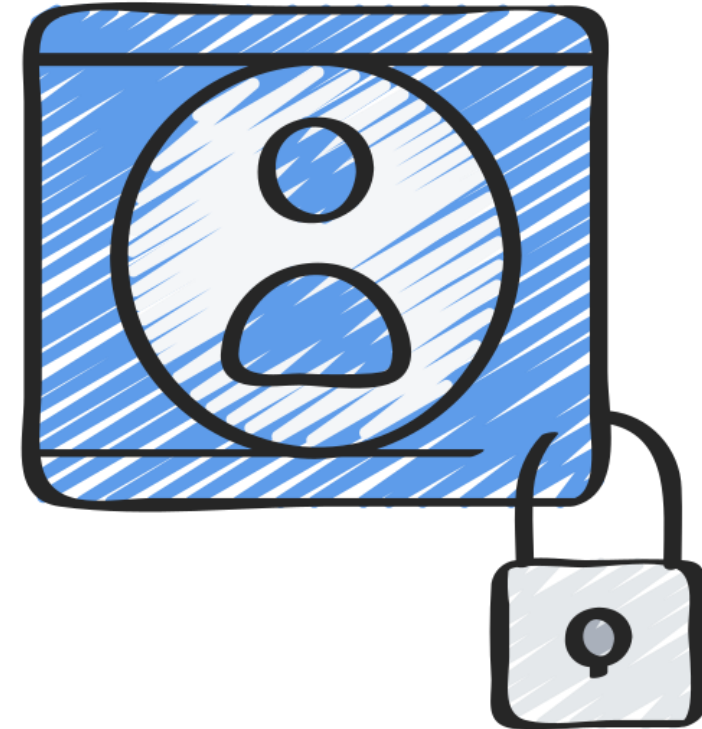
- ADD\_VOICEMAIL
- USE\_SIP
- PROCESS\_OUTGOING\_CALLS
- SEND\_SMS
- RECEIVE\_SMS
- READ\_SMS
- RECEIVE\_WAP\_PUSH
- RECEIVE\_MMS
- READ\_EXTERNAL\_STORAGE
- WRITE\_EXTERNAL\_STORAGE
- WRITE\_CALL\_LOG

# Special permissions

- **Special permissions** correspond to particular app operations. Only the platform and OEMs can define special permissions.
- **The platform** and **OEMs** usually define special permissions when they want to protect access to particularly powerful actions.
- The **Special app access** page in system settings contains a set of user-toggable operations. Many of these operations are implemented as special permissions.

# List Special permissions

- ACCESS\_NOTIFICATION\_POLICY
- BIND\_REMOTEVIEWS
- BIND\_TEXT\_SERVICE
- CHANGE\_CONFIGURATION
- CHANGE\_WIFI\_MULTICAST\_STATE
- CLEAR\_APP\_CACHE
- DELETE\_CACHE\_FILES
- INSTALL\_SHORTCUT
- KILL\_BACKGROUND\_PROCESSES
- READ\_SYNC\_SETTINGS
- SET\_ALARM
- SET\_WALLPAPER
- SET\_TIME\_ZONE
- UPDATE\_DEVICE\_STATS



# Manifest Elements

# What is Manifest ?

- Every app project must have an AndroidManifest.xml file, at the root of the project source set.
- The manifest file describes essential information about your app to the [Android build tools](#), the [Android operating system](#), and [Google Play](#).
- The manifest file is required to declare the following:
  - ✓ The components of the app, including all [activities](#), [services](#), [broadcast receivers](#), and [content providers](#).
  - ✓ The permissions that the app needs in order to access protected parts of the system or other apps.
  - ✓ The [hardware](#) and [software](#) features the app requires, which affects which devices can install the app from Google Play.

# Declare file features

For each app component , declare a corresponding XML element in the manifest file:

- <activity> for each subclass of Activity
- <service> for each subclass of Service
- <receiver> for each subclass of BroadcastReceiver
- <provider> for each subclass of ContentProvider

```
<manifest ... >
  <application ... >
    <activity android:name=".MainActivity" ... >
      ...
    </activity>
  </application>
</manifest>
```

# Declare permissions

- Android apps must request permission to access sensitive user data, such as **contacts** and **SMS**, or **certain system features**, such as the **camera** and **internet access**. Each permission is identified by a unique label in the manifest.

```
<manifest ... >
    <uses-permission android:name="android.permission.SEND_SMS"/>
    ...
</manifest>
```



# Declare device compatibility

- The manifest file is also where you can declare what types of hardware or software features your app requires and, by extension, which types of devices your app is compatible with.
- The <uses-feature> element lets you declare hardware and software features your app needs.

```
<manifest ... >
  <uses-feature android:name="android.hardware.sensor.compass"
               android:required="true" />
  ...
</manifest>
```

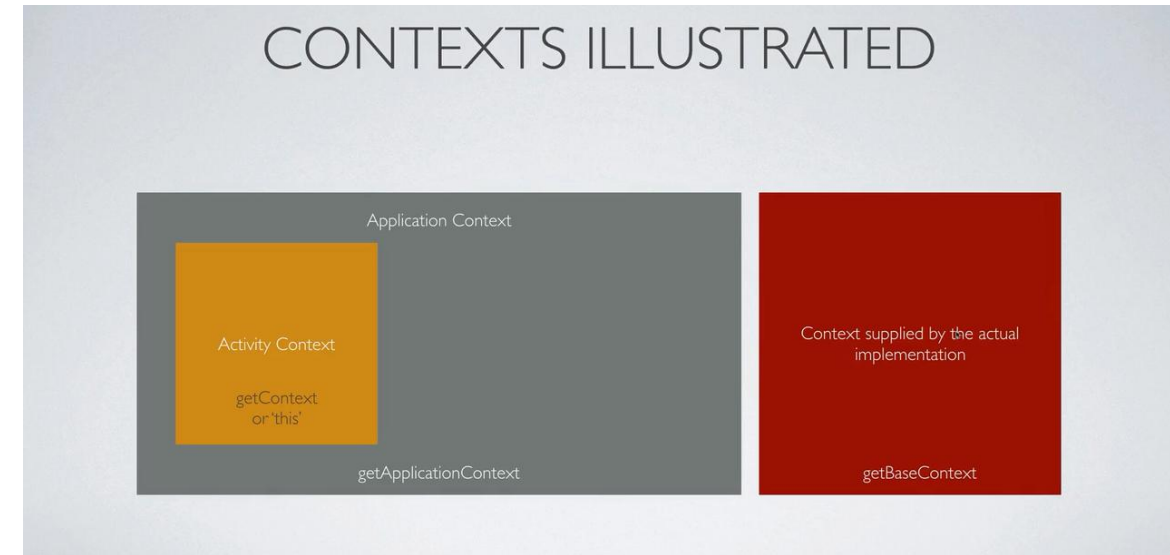
- App activities, services, and broadcast receivers are activated by *intents*.
- An intent is a message defined by an Intent object that describes an action to perform, including the data to be acted on, the category of component that is expected to perform the action, and other instructions.
- When an app issues an intent to the system, the system locates an app component that can handle the intent based on *intent filter* declarations in each app's manifest file.
- An app component can have any number of intent filters (defined with the <intent-filter> element), each one describing a different capability of that component.



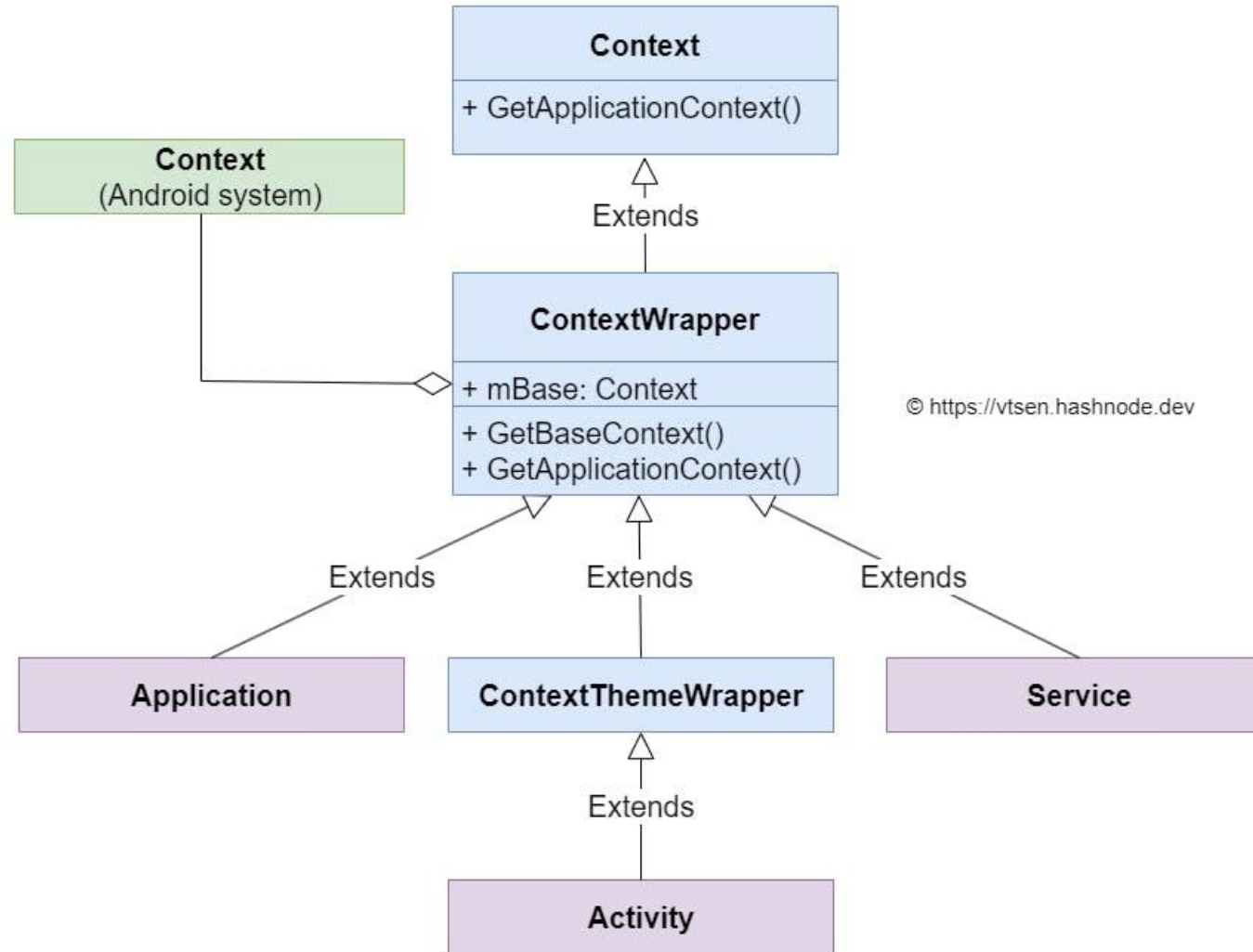
# Context

# What's context ?

- **Context** is the context of the current state of the application/object.
- **Context** is called to get information regarding another part of your program (activity and package/application).
- Can get **Context** by invoking *getApplicationContext()*, *getContext()*, *getBaseContext()* or *this*.



# Context with class diagram



# Typical uses of context

- **Creating new objects:** Creating new views, adapter, listener.

```
TextView tv = new TextView(getContext());  
ListAdapter adapter = new SimpleCursorAdapter(getApplicationContext(), ...);
```

- **Accessing standard common resource:** SharedPreferences , Services like LAYOUT\_INFLATER\_SERVICE.

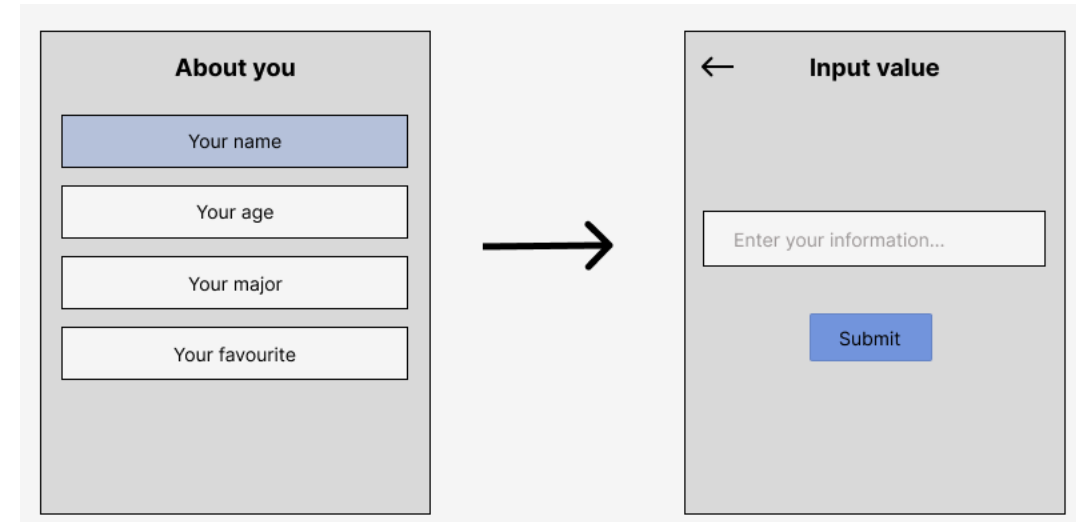
```
context.getSystemService(LAYOUT_INFLATER_SERVICE)  
getApplicationContext().getSharedPreferences(*name*, *mode*);
```

- **Accessing components implicitly:** Regarding content providers, broadcasts, intent.

```
getApplicationContext().getContentResolver().query(uri, ...);
```

# Assignment

- There are 2 activity, “About you” activity contains 4 items. When user click into each item, move to “Input value” activity.
- At “Input value” activity, user will enter your information that matches with selected item in “About you” activity. Validate value in Edittext, if It is valid, “Submit” button is clickable.
- After submit value in “Input value” activity, go back “About you” activity and fill value in selected item.
- **Bonus: Good UI is a plus point**



# THANK YOU!

